

Documento Técnico — Actividad Evaluativa 20%

Asignatura: Desarrollo de Aplicaciones Web
Estudiante: Isaias Sánchez
Fecha: 18 de mayo de 2026
Proyecto: API REST — Biblioteca Personal

Tema 1: Ensayo — Riesgos del Vibe Coding en el Desarrollo Profesional de Software

El *vibe coding* es una práctica emergente en la que el desarrollador delega la generación del código casi en su totalidad a herramientas de inteligencia artificial, guiándose por intuición, prompts en lenguaje natural y resultados rápidos, sin un análisis profundo del código producido. Aunque esta metodología democratiza el acceso al desarrollo de software y acelera la creación de prototipos, su adopción acrítica en entornos profesionales conlleva riesgos significativos que merecen una reflexión seria.

El primero de estos riesgos es la **dependencia cognitiva y la erosión del criterio técnico**. Cuando un desarrollador acepta sistemáticamente el código generado por IA sin comprender su funcionamiento interno, pierde la capacidad de razonarlo, depurarlo y adaptarlo ante situaciones imprevistas. La comprensión profunda de los algoritmos, las estructuras de datos y los patrones arquitectónicos no es un lujo académico: es la diferencia entre un profesional que resuelve problemas nuevos y uno que solo puede ejecutar recetas conocidas. Un programador que nunca entiende por qué funciona su código no puede garantizar que seguirá funcionando mañana.

El segundo riesgo es el de **seguridad y calidad del código**. Los modelos de lenguaje generan código estadísticamente plausible, no necesariamente correcto ni seguro. Es común que el código generado por IA contenga vulnerabilidades como inyecciones SQL, manejo inadecuado de errores, condiciones de carrera o exposición de datos sensibles, especialmente cuando el prompt no especifica explícitamente los requisitos de seguridad. Un desarrollador que no comprende los vectores de ataque (XSS, CSRF, RCE, entre otros) no puede reconocer si el código producido los introduce. El resultado es software que parece funcionar pero que crea superficies de ataque significativas en producción.

El tercer riesgo es el de la **deuda técnica invisible**. El código generado por IA tiende a resolver el problema inmediato de la manera más directa posible, sin considerar la mantenibilidad a largo plazo, la coherencia con la arquitectura existente ni las convenciones del equipo. Esta deuda técnica se acumula silenciosamente y se vuelve evidente solo cuando el sistema necesita escalar, cuando hay que incorporar un nuevo desarrollador al proyecto, o cuando un cambio de requisitos requiere refactorizar una base de código que nadie comprende del todo.

El *vibe coding*, utilizado como herramienta de asistencia y aceleración por un desarrollador competente, es un recurso valioso. El peligro no está en la herramienta sino en la disposición a substituir el pensamiento crítico por la comodidad de la generación automática. La formación técnica sólida es, precisamente, lo que convierte a un usuario de IA en un profesional del software capaz de aprovechar estas herramientas sin quedar subordinado a ellas.

Tema 2: Cuadro Comparativo — Monolitos vs Microservicios

Dimensión	Arquitectura Monolítica	Arquitectura de Microservicios
Definición	Aplicación única donde todos los módulos (UI, lógica de negocio, acceso a datos) están integrados en un solo proceso desplegable	Conjunto de servicios pequeños e independientes, cada uno ejecutando su propia lógica de negocio y comunicándose vía APIs (REST, gRPC, mensajes)
Despliegue	Un solo artefacto desplegable; simple al inicio pero complejo cuando crece	Cada servicio se despliega de forma independiente; mayor complejidad operacional inicial pero mayor flexibilidad

Escalabilidad	Escala el sistema completo (vertical u horizontal), incluso si solo un módulo tiene alta demanda	Escala únicamente los servicios que lo necesitan; más eficiente en recursos
Tecnología	Un solo stack tecnológico por toda la aplicación	Cada servicio puede usar el lenguaje, framework y base de datos más adecuado para su función
Comunicación	Llamadas directas en memoria entre módulos; rápido y simple	Comunicación por red (latencia, posibles fallos); requiere diseño cuidadoso de contratos
Consistencia de datos	Base de datos única; transacciones ACID simples	Cada servicio gestiona su propia base de datos; consistencia eventual, sagas para transacciones distribuidas
Curva de aprendizaje	Baja para equipos pequeños; todo el código está en un lugar	Alta; requiere conocimiento de contenedores, orquestación (Kubernetes), service mesh, distributed tracing
Tolerancia a fallos	Un fallo en un módulo puede tumbar toda la aplicación	Los fallos se aíslan por servicio; circuit breakers y retries limitan el impacto
Testing	Tests de integración simples al tener un solo proceso	Testing más complejo; requiere mocks y contract testing entre servicios
Velocidad inicial	Alta; ideal para MVPs y proyectos en etapa temprana	Baja inicialmente; la infraestructura necesaria (CI/CD por servicio, observabilidad) demora el primer deploy
Equipos	Adecuado para equipos pequeños y co-ubicados	Facilita equipos grandes y distribuidos; cada equipo es dueño de su servicio (Conway's Law)
Casos de uso ideales	Startups, MVPs, aplicaciones con tráfico predecible y equipo reducido	Plataformas de alto tráfico con equipos grandes (Netflix, Amazon, Uber); cuando distintos módulos tienen perfiles de carga muy diferentes

Conclusión: No existe una arquitectura universalmente superior. La decisión debe basarse en el tamaño del equipo, la madurez del producto, los requisitos de escala y la capacidad operacional disponible. Comenzar con un monolito bien estructurado y migrar progresivamente a microservicios cuando el crecimiento lo justifique (el llamado *Strangler Fig Pattern*) suele ser la estrategia más pragmática.

Tema 3: Investigación — Vectores de Ataque Web y Supply Chain Attacks

3.1 Vectores de Ataque Web

Cross-Site Scripting (XSS)

XSS ocurre cuando un atacante logra inyectar código JavaScript malicioso en páginas web que otros usuarios visualizan. Existen tres variantes: reflejado (el payload viaja en la URL y se ejecuta en la respuesta inmediata), almacenado (el payload se guarda en la base de datos y se ejecuta cada vez que se carga la página) y basado en DOM (la manipulación ocurre en el cliente sin pasar por el servidor). El impacto incluye robo de cookies de sesión, redireccionamiento a sitios de phishing, keylogging y defacement. La mitigación principal es la codificación de salida (HTML encoding) y el uso de Content Security Policy (CSP) para restringir las fuentes de scripts permitidas.

Cross-Site Request Forgery (CSRF)

CSRF explota la confianza que un sitio tiene en el navegador del usuario autenticado. El atacante induce a la víctima

a ejecutar una petición no deseada (por ejemplo, transferir fondos o cambiar contraseñas) aprovechando que el navegador envía automáticamente las cookies de sesión con cada request al dominio objetivo. La defensa estándar es el uso de tokens CSRF (valores aleatorios únicos por sesión y por formulario) que el servidor valida antes de procesar operaciones de cambio de estado. El atributo `SameSite` en cookies (configurado como `Strict` o `Lax`) también mitiga la mayoría de los vectores CSRF modernos.

Server-Side Request Forgery (SSRF)

En SSRF, el atacante hace que el servidor realice peticiones HTTP a destinos arbitrarios, incluyendo la red interna de la organización. Esto puede exponer servicios internos no públicos, credenciales de metadatos en la nube (como el endpoint `http://169.254.169.254` en AWS/GCP) y datos sensibles de infraestructura. La mitigación requiere validar y sanitizar todas las URLs proporcionadas por el usuario, usar listas blancas de destinos permitidos y aislar los servicios que realizan peticiones externas de los sistemas internos mediante segmentación de red.

SQL Injection

La inyección SQL permite a un atacante manipular las consultas a la base de datos al insertar fragmentos SQL maliciosos en campos de entrada. Un atacante puede extraer tablas enteras, modificar o eliminar datos, escalar privilegios o en algunos motores ejecutar comandos del sistema operativo. La defensa fundamental son las **consultas parametrizadas** (prepared statements) o los ORMs que las implementen por defecto, que separan los datos del código SQL estructuralmente. Nunca debe construirse SQL concatenando strings con input del usuario.

Remote Code Execution (RCE)

RCE es la capacidad de ejecutar código arbitrario en el servidor objetivo. Es generalmente el resultado más grave de una cadena de vulnerabilidades: deserialización insegura de objetos (Java, PHP), inyección de comandos en funciones como `exec()` o `system()` con input no sanitizado, o vulnerabilidades en dependencias (Log4Shell en 2021 permitió RCE con un simple string en un log). La mitigación es multicapa: sanitización estricta de inputs, principio de mínimo privilegio en la ejecución de procesos, WAF, y actualización proactiva de dependencias.

3.2 Supply Chain Attacks en npm y Composer

Los ataques a la cadena de suministro de software apuntan a los ecosistemas de dependencias que prácticamente todas las aplicaciones modernas consumen. En lugar de atacar directamente una aplicación bien protegida, el atacante compromete una librería que dicha aplicación incluye, propagando código malicioso a miles o millones de proyectos simultáneamente.

Vectores de ataque en npm (Node.js)

- *Typosquatting*: publicar paquetes con nombres similares a los populares (por ejemplo `lodash` vs `lodaysh`, `crossenv` vs `cross-env`). Un desarrollador que comete un typo al instalar una dependencia obtiene código malicioso.
- *Dependency confusion*: si una empresa usa paquetes internos con nombres no publicados en npm, un atacante puede publicar un paquete con el mismo nombre en el registro público. npm priorizará la versión pública más reciente sobre la privada si la configuración no lo previene.
- *Account takeover*: comprometer la cuenta npm de un mantenedor (mediante phishing o robo de tokens) y publicar una versión maliciosa de un paquete legítimo. El caso `event-stream` (2018) es un ejemplo clásico: un atacante ganó acceso como mantenedor de un paquete con millones de descargas semanales e introdujo código para robar wallets de Bitcoin.
- *Malicious packages at install*: usar los scripts `preinstall`, `install` o `postinstall` del `package.json` para ejecutar código arbitrario en el momento en que el desarrollador corre `npm install`.

Vectores de ataque en Composer (PHP)

Los vectores son análogos: typosquatting en Packagist, account takeover de mantenedores, y el uso de scripts `post-install-cmd` en `composer.json`. Composer también es susceptible a ataques donde un paquete malicioso se introduce como dependencia transitiva de una dependencia directa legítima.

Mitigaciones

1. **Lock files:** usar `package-lock.json` (npm) y `composer.lock` para fijar versiones exactas y hashes de integridad. Nunca ignorar los lock files en el repositorio.
 2. **Auditoría regular:** `npm audit` y `composer audit` detectan vulnerabilidades conocidas en dependencias actuales.
 3. **Scoping de permisos:** en npm, usar herramientas como `socket.dev` o GitHub's Dependabot para revisar cambios de comportamiento en actualizaciones de paquetes.
 4. **Principio de mínima dependencia:** evaluar si una librería es realmente necesaria antes de añadirla, especialmente si reemplaza pocas líneas de código propio.
 5. **Verificar popularidad y mantenimiento:** preferir paquetes con historial de mantenimiento activo, muchos colaboradores y repositorios públicos auditables.
 6. **Ambientes de CI/CD aislados:** ejecutar `npm install` en entornos sin acceso a producción para contener el radio de daño de un supply chain attack.
-

Documento generado como parte de la actividad evaluativa de la asignatura.

Repositorio GitHub: <https://github.com/isaias-sanchez/api-biblioteca-personal>

Demo en vivo: <https://api-biblioteca-personal-six.vercel.app>